

PySpark Mastery Guide

for Azure Data Engineers

All Topics · Concepts · Commands

Apache Spark

Azure Databricks

Delta Lake

ADF

TABLE OF CONTENTS

CH	Chapter	Key Topics
01	PySpark Fundamentals & Architecture	Core concepts, SparkContext, SparkSession
02	RDDs – Resilient Distributed Datasets	Transformations, Actions, Persistence
03	DataFrames & Spark SQL	Schema, queries, joins, aggregations
04	Spark Streaming & Structured Streaming	DStreams, watermarks, triggers, sinks
05	Performance Optimization	Partitioning, caching, broadcast, AQE
06	Delta Lake on Azure Databricks	ACID, Z-Ordering, CDF, Optimize
07	Azure Databricks Integration	Clusters, ADLS, AKV, Unity Catalog
08	Azure Data Factory with PySpark	Pipelines, Linked Services, Mappings
09	Data Quality & Testing	Great Expectations, Unit testing, DQ
10	Security, Governance & Best Practices	RBAC, encryption, lineage, patterns

This guide covers every topic, concept and PySpark command required to become a proficient Azure Data Engineer — from core Spark architecture through production-grade patterns on Databricks and Azure.

PySpark Fundamentals & Architecture

Apache Spark is a unified analytics engine for large-scale data processing. PySpark is the Python API for Spark, enabling you to write distributed programs in Python. Understanding the architecture is the foundation of everything else.

1.1 Core Architecture Concepts

- **Driver Program** – orchestrates the execution; runs `main()` and creates `SparkContext`
- **Cluster Manager** – allocates resources (YARN, Mesos, Kubernetes, Standalone, Databricks)
- **Worker Nodes** – machines that run Executor processes
- **Executors** – JVM processes on workers; execute tasks and store data in cache
- **Tasks** – smallest unit of work sent to an Executor
- **Jobs / Stages / Tasks** – hierarchy created from an Action call
- **DAG Scheduler** – builds a directed acyclic graph of stages
- **Task Scheduler** – assigns tasks to executors, handles retries
- **Shuffle** – data exchange between partitions (expensive; minimise it)

1.2 SparkSession & SparkContext

`SparkSession` is the single entry point in Spark 2+. `SparkContext (sc)` is accessible via `spark.sparkContext`.

```
from pyspark.sql import SparkSession spark = (SparkSession.builder .appName("AzureDE_App")
.master("local[*]") # local or yarn / databricks .config("spark.executor.memory", "4g")
.config("spark.executor.cores", "2") .config("spark.sql.shuffle.partitions", "200")
.enableHiveSupport() # optional - for Hive metastore .getOrCreate()) sc = spark.sparkContext #
SparkContext sql = spark.sql # shortcut for spark.sql(...) spark.stop() # always stop when done
```

1.3 Key Configuration Properties

Property	Purpose
<code>spark.executor.memory</code>	Heap memory per executor (e.g. 4g)
<code>spark.executor.cores</code>	CPU cores per executor
<code>spark.driver.memory</code>	Driver heap memory
<code>spark.sql.shuffle.partitions</code>	Partitions after a shuffle (default 200)
<code>spark.default.parallelism</code>	Default RDD partition count
<code>spark.serializer</code>	KryoSerializer for better performance
<code>spark.sql.adaptive.enabled</code>	Enable Adaptive Query Execution (AQE)
<code>spark.dynamicAllocation.enabled</code>	Auto-scale executors
<code>spark.sql.files.maxPartitionBytes</code>	Max bytes per file partition (128 MB default)

1.4 Execution Model

- **Lazy Evaluation** – transformations are NOT executed until an action is called

- **Lineage** – Spark tracks the series of transformations (DAG) for fault tolerance
 - **Narrow Transformations** – no shuffle; e.g. map, filter, union
 - **Wide Transformations** – require shuffle; e.g. groupBy, join, distinct
 - **Pipelining** – Spark combines narrow transformations into a single stage
- Always call `spark.sql.shuffle.partitions = 2-3× your core count for optimal performance on your cluster size`.

RDDs – Resilient Distributed Datasets

RDDs are the low-level distributed data structure. While DataFrames are preferred, RDDs are essential for unstructured data, fine-grained control, and understanding Spark internals.

2.1 Creating RDDs

```
# From a Python collection rdd = sc.parallelize([1, 2, 3, 4, 5], numSlices=4) # From a file
(ADLS / DBFS / local) rdd =
sc.textFile("abfss://container@storage.dfs.core.windows.net/data/*.csv") # From existing RDD
rdd2 = rdd.map(lambda x: x * 2) # From DataFrame rdd = df.rdd # Row objects rdd =
df.rdd.map(lambda r: r.asDict()) # dicts
```

2.2 Transformations (Lazy)

Transformation	Description
map(func)	Apply func to each element → new RDD
flatMap(func)	Like map but flattens results
filter(func)	Keep elements where func returns True
distinct()	Remove duplicates
union(rdd)	Combine two RDDs (duplicates kept)
intersection(rdd)	Common elements of two RDDs
subtract(rdd)	Elements in first RDD not in second
reduceByKey(func)	Aggregate values by key (combiner on map side)
groupByKey()	Group values by key (all data shuffled – avoid)
sortByKey(asc=True)	Sort by key
mapPartitions(func)	Apply func to whole partition (efficient I/O)
mapPartitionsWithIndex(f)	Like mapPartitions with partition index
repartition(n)	Reshuffle to n partitions (increases or decreases)
coalesce(n)	Reduce partitions without full shuffle
sample(replace, frac, seed)	Random sample
zip(rdd)	Zip two RDDs into (k,v) pairs
join(rdd)	Inner join two pair RDDs by key
leftOuterJoin(rdd)	Left outer join
cogroup(rdd)	Group values from multiple RDDs by key

2.3 Actions (Trigger Execution)

Action	Returns
collect()	All elements as Python list (caution: driver OOM)
count()	Number of elements
first()	First element
take(n)	First n elements
takeSample(replace, n, seed)	Random n elements
top(n)	Top n elements (descending)
reduce(func)	Aggregate elements with associative func
fold(zeroVal, func)	Like reduce with a zero value
aggregate(zeroVal, seqOp, combOp)	Aggregate with different return type
foreach(func)	Apply func to each element (side effects)
saveAsTextFile(path)	Save each element as a string to files
saveAsSequenceFile(path)	Save as Hadoop SequenceFile
countByKey()	Count per key as dict
collectAsMap()	Collect (k,v) RDD as Python dict

2.4 Persistence / Caching

```
from pyspark import StorageLevel rdd.cache() # MEMORY_AND_DISK
rdd.persist(StorageLevel.MEMORY_ONLY) rdd.persist(StorageLevel.MEMORY_ONLY_2) # 2 replicas
rdd.persist(StorageLevel.DISK_ONLY) rdd.persist(StorageLevel.OFF_HEAP) # Tungsten off-heap
rdd.unpersist() # release cache
```

- **MEMORY_ONLY** – fastest; spills to recompute if OOM
- **MEMORY_AND_DISK** – spills to disk; good default
- **DISK_ONLY** – use when data is large and recompute is expensive
- **_2 suffix** – 2 replicas for fault tolerance

■ **Prefer DataFrame/Dataset caching over RDD; the Catalyst optimizer works better.**

DataFrames & Spark SQL

DataFrames are the primary abstraction for structured data in PySpark. They provide a rich set of optimised operations and are the recommended API for most data engineering work.

3.1 Creating DataFrames

```
# From CSV / JSON / Parquet / Delta / ORC / Avro df = spark.read.csv("path/*.csv", header=True,
inferSchema=True) df = spark.read.json("path/*.json", multiLine=True) df =
spark.read.parquet("path/*.parquet") df = spark.read.format("delta").load("path/") df =
spark.read.format("avro").load("path/") df = spark.read.orc("path/*.orc") # With explicit schema
from pyspark.sql.types import * schema = StructType([ StructField("id", LongType(),
nullable=False), StructField("name", StringType(), nullable=True), StructField("value",
DoubleType(), nullable=True), StructField("ts", TimestampType(), nullable=True), ]) df =
spark.read.schema(schema).csv("path/data.csv", header=True) # From Python list / Pandas df =
spark.createDataFrame([(1,"Alice",3.5),(2,"Bob",4.1)], ["id","name","score"]) import pandas as
pd pdf = pd.DataFrame({"a":[1,2],"b":[3,4]}) df = spark.createDataFrame(pdf)
```

3.2 DataFrame Transformations

```
from pyspark.sql import functions as F from pyspark.sql.window import Window # Column selection
& renaming df.select("col1", "col2") df.select(F.col("col1").alias("c1"), F.expr("col2 * 2 as
c2")) df.withColumnRenamed("old", "new") df.drop("col1", "col2") # Filtering
df.filter(F.col("age") > 18) df.where((F.col("status") == "active") &
F.col("score").isNotNull()) # Adding / modifying columns df.withColumn("full_name",
F.concat_ws(" ", "first","last")) df.withColumn("year", F.year(F.col("ts")))
df.withColumn("flag", F.when(F.col("val") > 0, "pos").otherwise("neg")) # Type casting
df.withColumn("id", F.col("id").cast(LongType())) # Sorting df.orderBy("col1",
F.col("col2").desc()) df.sort(F.col("score").desc_nulls_last()) # Distinct / dedup df.distinct()
df.dropDuplicates(["id", "date"]) # Sampling df.sample(fraction=0.1, seed=42) df.limit(100) #
Repartition df.repartition(200) # reshuffle df.repartition(200, "country") # partition by column
df.coalesce(10) # reduce without shuffle
```

3.3 Aggregations

```
# Basic aggregations df.groupBy("dept").agg( F.count("*").alias("cnt"),
F.sum("salary").alias("total_sal"), F.avg("salary").alias("avg_sal"),
F.max("salary").alias("max_sal"), F.min("salary").alias("min_sal"),
F.countDistinct("emp_id").alias("unique_emps"), F.collect_list("name").alias("names"),
F.collect_set("skill").alias("skills"), F.stddev("salary").alias("std"),
F.variance("salary").alias("var"), F.percentile_approx("salary", 0.5).alias("median"),
F.first("manager").alias("mgr"), F.last("status").alias("last_status"), ) # Pivot
df.groupBy("dept").pivot("year", [2022,2023,2024]).sum("revenue") # Cube / Rollup
df.cube("region","dept").agg(F.sum("sales")) df.rollup("region","dept").agg(F.sum("sales")) #
Global aggregation df.agg(F.sum("salary"), F.avg("salary"))
```

3.4 Joins

```
# Join types: inner, left, right, outer, left_semi, left_anti, cross df1.join(df2, on="id",
how="inner") df1.join(df2, on=["id","date"], how="left") df1.join(df2, df1["id"] ==
df2["emp_id"], "outer") df1.join(df2, on="id", how="left_semi") # keys in df1 that exist in df2
df1.join(df2, on="id", how="left_anti") # keys in df1 NOT in df2 df1.crossJoin(df2) # cartesian
(be careful!) # Broadcast join (small table hint) from pyspark.sql.functions import broadcast
df_large.join(broadcast(df_small), "key")
```

■ Use broadcast joins when one table fits in memory (<200 MB). Set `spark.sql.autoBroadcastJoinThreshold = 50MB` (default).

3.5 Window Functions

```
from pyspark.sql.window import Window w_dept = Window.partitionBy("dept").orderBy("salary")
w_dept_rows = Window.partitionBy("dept").orderBy("salary").rowsBetween(-2, 0) w_dept_range =
Window.partitionBy("dept").orderBy("salary").rangeBetween(-1000, 0) df.withColumn("rank",
F.rank().over(w_dept)) df.withColumn("dense_rank", F.dense_rank().over(w_dept))
df.withColumn("row_number", F.row_number().over(w_dept)) df.withColumn("ntile",
F.ntile(4).over(w_dept)) # quartile df.withColumn("pct_rank", F.percent_rank().over(w_dept))
df.withColumn("cume_dist", F.cume_dist().over(w_dept)) df.withColumn("lag_sal", F.lag("salary",
1, 0).over(w_dept)) df.withColumn("lead_sal", F.lead("salary",1, 0).over(w_dept))
df.withColumn("run_sum", F.sum("salary").over(w_dept_rows)) df.withColumn("run_avg",
F.avg("salary").over(w_dept_rows)) df.withColumn("first_val", F.first("salary").over(w_dept))
df.withColumn("last_val", F.last("salary").over(w_dept))
```

3.6 String, Date & Array Functions

```
# String F.upper("col"), F.lower("col"), F.trim("col"), F.ltrim("col"), F.rtrim("col")
F.substring("col", 1, 5), F.instr("col","sub"), F.length("col")
F.regexp_replace("col","pattern","repl"), F.regexp_extract("col","(\\d+)",1) F.split("col",""),
F.concat("a","b"), F.concat_ws("-", "a","b") F.lpad("col",10,"0"), F.rpad("col",10," ")
F.translate("col","abc","ABC"), F.initcap("col"), F.format_string("%05d","col") # Date & Time
F.current_date(), F.current_timestamp(), F.now() F.year("ts"), F.month("ts"),
F.dayofmonth("ts"), F.hour("ts") F.date_add("date",7), F.date_sub("date",7),
F.datediff("d1","d2") F.months_between("d1","d2"), F.add_months("date",3)
F.to_date("col","yyyy-MM-dd"), F.to_timestamp("col","yyyy-MM-dd HH:mm:ss")
F.date_format("ts","yyyy/MM/dd"), F.trunc("ts","month") F.from_unixtime(F.col("epoch")),
F.unix_timestamp("ts","yyyy-MM-dd") F.from_utc_timestamp("ts","Asia/Kolkata") # Array & Map
F.array("c1","c2"), F.array_contains("arr","val"), F.array_distinct("arr")
F.array_union("a1","a2"), F.array_intersect("a1","a2"), F.array_except("a1","a2")
F.flatten("arr"), F.array_sort("arr"), F.reverse("arr") F.size("arr"), F.slice("arr",2,4),
F.element_at("arr",1) F.explode("arr"), F.posexplode("arr"), F.explode_outer("arr")
F.map_keys("map"), F.map_values("map"), F.map_from_arrays("k","v")
F.create_map("k1","v1","k2","v2") # Struct F.struct("a","b","c"), F.col("struct_col.field")
F.to_json(F.struct("")), F.from_json("json_col", schema) F.get_json_object("col","$.key"),
F.json_tuple("col","k1","k2")
```

3.7 Spark SQL

```
# Register temp views df.createOrReplaceTempView("orders")
df.createOrReplaceGlobalTempView("orders_global") # available across sessions # Run SQL result =
spark.sql('' SELECT dept, SUM(salary) AS total, AVG(salary) AS avg, RANK() OVER (ORDER BY
SUM(salary) DESC) AS rank FROM employees WHERE status = 'active' GROUP BY dept HAVING
SUM(salary) > 100000 ORDER BY total DESC '') # Useful SQL commands via spark.sql
spark.sql("SHOW DATABASES") spark.sql("USE mydb") spark.sql("SHOW TABLES") spark.sql("DESCRIBE
TABLE orders") spark.sql("DESCRIBE EXTENDED orders") spark.sql("EXPLAIN SELECT * FROM orders
WHERE id = 1") spark.sql("ANALYZE TABLE orders COMPUTE STATISTICS")
```

3.8 Writing DataFrames

```
# Write modes: overwrite, append, ignore, error (default)
df.write.mode("overwrite").parquet("path/out/")
df.write.mode("append").format("delta").save("path/delta/")
df.write.mode("overwrite").option("header","true").csv("path/csv/")
```



```
df.write.mode("overwrite").json("path/json/") df.write.mode("overwrite").orc("path/orc/") #
Partition by column df.write.mode("overwrite").partitionBy("year", "month").parquet("path/") #
Save as table df.write.mode("overwrite").saveAsTable("catalog.schema.table") # JDBC
df.write.jdbc(url="jdbc:sqlserver://...", table="dbo.orders", mode="overwrite",
properties={"user": "admin", "password": "xxx",
"driver": "com.microsoft.sqlserver.jdbc.SQLServerDriver"}) # Repartition before write for file
count control df.repartition(10).write.mode("overwrite").parquet("path/")
df.coalesce(1).write.mode("overwrite").csv("path/single_file/")
```

Spark Streaming & Structured Streaming

Structured Streaming is the recommended streaming API – a continuous incremental query on an unbounded DataFrame backed by a fault-tolerant checkpoint.

4.1 Reading Streams

```
# Kafka source stream_df = (spark.readStream .format("kafka") .option("kafka.bootstrap.servers",
"broker:9092") .option("subscribe", "topic1,topic2") .option("startingOffsets", "earliest") # or
latest / json offsets .option("maxOffsetsPerTrigger", 10000) .load()) # Parse Kafka value from
pyspark.sql.types import * schema = StructType([StructField("event", StringType()),
StructField("ts", TimestampType())]) parsed = (stream_df .selectExpr("CAST(value AS STRING) as
json_str") .select(F.from_json("json_str", schema).alias("data")) .select("data.*")) # Delta /
file source stream_df = (spark.readStream .format("delta") .option("maxFilesPerTrigger", 1000)
.load("abfss://container@storage.dfs.core.windows.net/events/")) # Rate source (testing)
spark.readStream.format("rate").option("rowsPerSecond", 100).load()
```

4.2 Streaming Operations

```
# Stateless: same ops as batch parsed.filter(F.col("event") == "click").withColumn("hr",
F.hour("ts")) # Aggregations with watermark agg = (parsed .withWatermark("ts", "10 minutes") #
late data tolerance .groupBy(F.window("ts","5 minutes","1 minute"), "event")
.agg(F.count("*").alias("cnt"))) # Deduplication dedup = (parsed .withWatermark("ts", "1 hour")
.dropDuplicates(["event_id", "ts"])) # Streaming joins (stream-stream, stream-static) static_df
= spark.read.delta("path/lookup/") enriched = parsed.join(broadcast(static_df), "user_id",
"left")
```

4.3 Writing Streams – Sinks & Triggers

```
from pyspark.sql.streaming import DataStreamWriter # Console sink (dev / debug) query =
(agg.writeStream.format("console") .outputMode("complete") .option("truncate", False)
.trigger(processingTime="30 seconds") .start()) # Delta sink (production) query =
(agg.writeStream .format("delta") .outputMode("append") # append / complete / update
.option("checkpointLocation", "abfss://cont@st.dfs.core.windows.net/chk/agg/")
.option("mergeSchema", "true") .trigger(processingTime="1 minute")
.start("abfss://cont@st.dfs.core.windows.net/output/agg/")) # Kafka sink query =
(parsed.selectExpr("CAST(event_id AS STRING) AS key", "to_json(struct(*)) AS value")
.writeStream.format("kafka") .option("kafka.bootstrap.servers", "broker:9092")
.option("topic", "output-topic") .option("checkpointLocation", "chk/kafka/")
.trigger(continuous="1 second") # Continuous processing mode .start()) # Foreach sink (custom)
def process_row(row): ... query = stream_df.writeStream.foreach(process_row).start() # Await /
manage query.awaitTermination() query.awaitTermination(timeout=60) query.stop() query.status,
query.lastProgress, query.recentProgress
```

4.4 Output Modes & Triggers

Mode / Trigger	Use When
outputMode('append')	Only new rows are written – no updates
outputMode('complete')	Entire result table rewritten each trigger
outputMode('update')	Only changed rows written

Mode / Trigger	Use When
trigger(processingTime='30s')	Micro-batch every 30 seconds
trigger(once=True)	Run once (like batch) – scheduled jobs
trigger(availableNow=True)	Process all available data then stop
trigger(continuous='1 second')	True continuous low-latency mode

Performance Optimization

Understanding how to tune PySpark jobs is critical for production Azure Data Engineering. Poor performance often stems from skew, excess shuffles, and misconfigurations.

5.1 Partitioning Strategy

- **Default partitions** – `spark.sql.shuffle.partitions = 200` (tune to 2-3× cores)
- **File partitions** – `spark.sql.files.maxPartitionBytes = 128MB`
- **repartition(n)** – use before wide operations or writes
- **coalesce(n)** – reduce partitions after filter; no full shuffle
- **partitionBy()** – write with partition pruning in mind (low cardinality cols)
- **Skew** – use salting, AQE skew join, or repartition on salted key

```
# Salt skewed key import pyspark.sql.functions as F SALT = 10 df_salted = df.withColumn("salt",
(F.rand()*SALT).cast("int")) .withColumn("salted_key", F.concat_ws("_", "key", "salt")) # AQE
settings spark.conf.set("spark.sql.adaptive.enabled", "true")
spark.conf.set("spark.sql.adaptive.coalescePartitions.enabled", "true")
spark.conf.set("spark.sql.adaptive.skewJoin.enabled", "true")
spark.conf.set("spark.sql.adaptive.advisoryPartitionSizeInBytes", "128m")
```

5.2 Caching & Persistence

```
df.cache() # MEMORY_AND_DISK – safe default df.persist(StorageLevel.MEMORY_ONLY)
df.persist(StorageLevel.DISK_ONLY) df.unpersist() # Check cache
spark.catalog.isCached("table_name") spark.catalog.uncacheTable("table_name") # Check storage
spark.sparkContext.statusTracker().getExecutorInfos() # Important: cache AFTER transformations,
BEFORE reuse filtered = df.filter(F.col("active") == True).cache() filtered.count() #
materialise cache # ... use filtered multiple times
```

5.3 Broadcast Variables & Accumulators

```
# Broadcast small lookup dict lookup = {"NY": "New York", "CA": "California"} bc =
sc.broadcast(lookup) df.withColumn("state_full", F.udf(lambda x:
bc.value.get(x, ""))(F.col("state"))) # Accumulator (counter / sum) counter = sc.accumulator(0)
def count_nulls(row): global counter if row["val"] is None: counter += 1
df.rdd.foreach(count_nulls) print(counter.value)
```

5.4 UDFs vs Built-in Functions

- **Avoid Python UDFs** – serialization overhead (Python ↔ JVM); use built-in F.* first
- **Pandas UDF (vectorized)** – much faster; uses Apache Arrow for batch transfer
- **SQL functions** – register for Spark SQL reuse

```
# Pandas UDF – Series → Series from pyspark.sql.functions import pandas_udf from
pyspark.sql.types import DoubleType import pandas as pd @pandas_udf(DoubleType()) def
score_udf(s: pd.Series) -> pd.Series: return s * 1.1 df.withColumn("new_score",
score_udf("score")) # Pandas UDF – grouped map (DataFrame → DataFrame) from
pyspark.sql.functions import PandasUDFType schema = df.schema @pandas_udf(schema,
PandasUDFType.GROUPED_MAP) def normalize(pdf: pd.DataFrame) -> pd.DataFrame: pdf["score"] =
(pdf["score"] - pdf["score"].mean()) / pdf["score"].std() return pdf
df.groupby("dept").apply(normalize)
```

5.5 Reading & Writing Optimizations

- **Predicate Pushdown** – filter early; Parquet/Delta skip row groups
- **Column Pruning** – select only needed columns
- **Partition Pruning** – filter on partition columns to skip files
- **File format** – prefer Parquet/Delta for columnar compression; avoid CSV in prod
- **Compaction** – small file problem: use Delta OPTIMIZE or repartition before write
- **Z-Ordering** – co-locate related data for multi-column filters (Delta)

```
# Enable predicate pushdown (default on) spark.conf.set("spark.sql.parquet.filterPushdown",
"true") spark.conf.set("spark.sql.orc.filterPushdown", "true") # Projection pushdown df =
spark.read.parquet("path/").select("id","name","ts") # only read 3 cols # Partition pruning df =
spark.read.parquet("path/").filter(F.col("year") == 2024) # reads year=2024/ only #
Auto-optimize writes (Databricks Delta)
spark.conf.set("spark.databricks.delta.optimizeWrite.enabled", "true")
spark.conf.set("spark.databricks.delta.autoCompact.enabled", "true")
```

Delta Lake on Azure Databricks

Delta Lake is the open-source storage layer on Databricks providing ACID transactions, schema enforcement, time travel, and scalable metadata handling. It is the cornerstone of the Lakehouse architecture.

6.1 Creating & Writing Delta Tables

```
# Write as Delta df.write.format("delta").mode("overwrite").save("abfss://cont@st.dfs.core.windows.net/delta/orders/") df.write.format("delta").mode("overwrite").partitionBy("year","month").save("abfss://cont@st.dfs.core.windows.net/delta/orders/") # Save as managed table in Unity Catalog df.write.format("delta").saveAsTable("catalog.schema.orders") # Create empty table with schema from delta.tables import DeltaTable DeltaTable.create(spark).tableName("catalog.schema.orders").addColumn("id", "LONG").addColumn("amount", "DOUBLE").addColumn("status", "STRING").partitionedBy("year", "month").execute()
```

6.2 DML – Merge / Update / Delete

```
from delta.tables import DeltaTable dt = DeltaTable.forPath(spark, "abfss://cont@st.dfs.core.windows.net/delta/orders/") # OR dt = DeltaTable.forName(spark, "catalog.schema.orders") # MERGE (upsert) (dt.alias("tgt").merge(df_updates.alias("src"), "tgt.id = src.id").whenMatchedUpdateAll().whenNotMatchedInsertAll().whenNotMatchedBySourceDelete() # SCD Type 1 delete .execute()) # Conditional merge (dt.alias("t").merge(df_new.alias("s"), "t.id = s.id").whenMatchedUpdate(condition="s.amount > t.amount", set={"amount":"s.amount","updated_at":"s.ts"}).whenNotMatchedInsert(values={"id":"s.id","amount":"s.amount"}).execute()) # UPDATE dt.update(condition="status = 'pending'", set={"status": "'processing'"}) # DELETE dt.delete(condition="status = 'cancelled' AND year < 2022") # SQL equivalents spark.sql("MERGE INTO catalog.schema.orders USING updates ON ...") spark.sql("UPDATE catalog.schema.orders SET status='done' WHERE id=1") spark.sql("DELETE FROM catalog.schema.orders WHERE year < 2020")
```

6.3 Time Travel

```
# Read by version df_v3 = spark.read.format("delta").option("versionAsOf", 3).load("abfss://cont@st.dfs.core.windows.net/delta/orders/") # Read by timestamp df_ts = spark.read.format("delta").option("timestampAsOf", "2024-01-15").load("path/") # Restore a table dt.restoreToVersion(3) dt.restoreToTimestamp("2024-01-15 00:00:00") # View history dt.history().show(20) dt.history(5).select("version","timestamp","operation","operationMetrics").show() # SQL spark.sql("DESCRIBE HISTORY catalog.schema.orders") spark.sql("SELECT * FROM catalog.schema.orders VERSION AS OF 3") spark.sql("SELECT * FROM catalog.schema.orders TIMESTAMP AS OF '2024-01-15'")
```

6.4 Optimize, Z-Order & Vacuum

```
# Optimize (compact small files) spark.sql("OPTIMIZE catalog.schema.orders") spark.sql("OPTIMIZE catalog.schema.orders WHERE year = 2024") # Z-Order (co-locate by query columns) spark.sql("OPTIMIZE catalog.schema.orders ZORDER BY (customer_id, order_date)") # Vacuum (remove old files; default retention 7 days) spark.sql("VACUUM catalog.schema.orders") spark.sql("VACUUM catalog.schema.orders RETAIN 168 HOURS") spark.sql("VACUUM catalog.schema.orders RETAIN 0 HOURS DRY RUN") # preview # Auto-optimize (Databricks) spark.sql("ALTER TABLE catalog.schema.orders SET TBLPROPERTIES ('delta.autoOptimize.optimizeWrite='true','delta.autoOptimize.autoCompact='true')")
```

6.5 Schema Evolution & Change Data Feed

```
# Schema evolution on write df.write.format("delta") .option("mergeSchema","true")
.mode("append").save("path/") # Schema enforcement (default - raises error on mismatch)
spark.conf.set("spark.databricks.delta.schema.autoMerge.enabled","true") # Change Data Feed
(CDF) spark.sql("ALTER TABLE orders SET TBLPROPERTIES (delta.enableChangeDataFeed = true)") #
Read CDF changes = (spark.read.format("delta") .option("readChangeFeed","true")
.option("startingVersion", 5) .table("catalog.schema.orders")) changes.filter("_change_type IN
('insert','update_postimage')).show()
```

Azure Databricks Integration

Azure Databricks is the managed Spark platform on Azure. Mastering its ecosystem is essential for production data engineering.

7.1 ADLS Gen2 Access Patterns

```
# Service Principal OAuth (preferred for production)
spark.conf.set("fs.azure.account.auth.type.<storage>.dfs.core.windows.net", "OAuth")
spark.conf.set("fs.azure.account.oauth.provider.type.<storage>.dfs.core.windows.net",
"org.apache.hadoop.fs.azurebfs.oauth2.ClientCredsTokenProvider")
spark.conf.set("fs.azure.account.oauth2.client.id.<storage>.dfs.core.windows.net",
"<client-id>") spark.conf.set("fs.azure.account.oauth2.client.secret.<storage>.dfs.core.windows.
net", "<secret>")
spark.conf.set("fs.azure.account.oauth2.client.endpoint.<storage>.dfs.core.windows.net",
"https://login.microsoftonline.com/<tenant-id>/oauth2/token") # Mount ADLS (legacy / DBFS)
dbutils.fs.mount( source="abfss://container@storage.dfs.core.windows.net/",
mount_point="/mnt/mydata", extra_configs={"fs.azure.account.auth.type...": "OAuth", ...}) #
Access Key Vault secrets (preferred) secret = dbutils.secrets.get(scope="kv-scope",
key="adls-secret") # Use ABFSS directly (no mount) df =
spark.read.parquet("abfss://container@storage.dfs.core.windows.net/data/")
```

7.2 Unity Catalog Commands

```
# Catalog / Schema / Table management spark.sql("CREATE CATALOG IF NOT EXISTS my_catalog")
spark.sql("CREATE SCHEMA IF NOT EXISTS my_catalog.bronze") spark.sql("CREATE TABLE
my_catalog.bronze.events USING DELTA LOCATION 'abfss://...') spark.sql("GRANT SELECT ON TABLE
my_catalog.bronze.events TO `group@domain.com`") spark.sql("GRANT MODIFY ON TABLE
my_catalog.bronze.events TO `de-team`") spark.sql("SHOW GRANTS ON TABLE
my_catalog.bronze.events") spark.sql("CREATE EXTERNAL LOCATION my_loc URL 'abfss://...' WITH
(STORAGE CREDENTIAL my_cred)") # Tags & comments spark.sql("ALTER TABLE my_catalog.bronze.events
SET TAGS ('env'='prod', 'pii'='false')") spark.sql("COMMENT ON TABLE my_catalog.bronze.events IS
'Raw clickstream events'") # Information schema spark.sql("SELECT * FROM
my_catalog.information_schema.tables WHERE table_schema='bronze'")
```

7.3 Cluster Configuration Best Practices

Setting	Recommendation
Cluster Mode	Standard (interactive) / Job (automated pipelines)
Runtime	Databricks Runtime ML for ML; LTS for stability
Autoscaling	Enable with min=2, max=10 workers for jobs
Spot Instances	Use for workers; on-demand for driver
Instance Pool	Pre-warm instances to reduce cluster start time
Init Scripts	Install custom libs via cluster init scripts
Cluster Policies	Enforce org standards (cost tags, configs)
Photon	Enable for SQL-heavy workloads on Databricks

7.4 Databricks Utilities (dbutils)

```
# File system dbutils.fs.ls("abfss://container@storage.dfs.core.windows.net/")
dbutils.fs.cp("abfss://.../src/", "abfss://.../dst/", recurse=True)
dbutils.fs.mv("src_path", "dst_path") dbutils.fs.rm("path/", recurse=True)
dbutils.fs.mkdirs("path/newdir/") # Secrets dbutils.secrets.listScopes()
dbutils.secrets.list("scope") dbutils.secrets.get(scope="kv-scope", key="my-secret") # Widgets
(parameterised notebooks) dbutils.widgets.text("start_date", "2024-01-01", "Start Date")
dbutils.widgets.dropdown("env", "prod", ["dev", "staging", "prod"]) val =
dbutils.widgets.get("start_date") dbutils.widgets.removeAll() # Notebook workflow
dbutils.notebook.run("./child_notebook", timeout_seconds=300, arguments={"param": "value"})
dbutils.notebook.exit("SUCCESS")
```

Azure Data Factory with PySpark

ADF orchestrates PySpark workloads. Key integration patterns include Databricks Notebook activities, Mapping Data Flows, and parameterised pipelines.

8.1 ADF + Databricks Integration

- **Databricks Notebook Activity** – run a notebook with parameters; capture exit value
- **Databricks JAR Activity** – submit a compiled JAR to a cluster
- **Databricks Python Activity** – run a .py script
- **Linked Service** – authenticate via Managed Identity or Service Principal
- **Job Cluster vs Interactive** – always use Job clusters in ADF for cost efficiency
- **Pipeline Parameters** – pass `@pipeline().parameters.start_date` to notebooks
- **Error Handling** – set retry policy; use If Condition on activity outcome

8.2 Passing Parameters from ADF to Databricks

```
# In ADF Databricks Notebook Activity → Settings → Base Parameters: # { "start_date":  
"@pipeline().parameters.start_date", # "env": "@pipeline().globalParameters.env" } # In the  
Notebook (receive parameters) dbutils.widgets.text("start_date","", "Start Date")  
dbutils.widgets.text("env","prod","Environment") start_date = dbutils.widgets.get("start_date")  
env = dbutils.widgets.get("env") # Return result to ADF  
dbutils.notebook.exit(json.dumps({"rows_processed": count, "status": "OK"}))
```

8.3 Parameterised PySpark Jobs Pattern

```
import argparse, sys from pyspark.sql import SparkSession def parse_args(): p =  
argparse.ArgumentParser() p.add_argument("--input", required=True) p.add_argument("--output",  
required=True) p.add_argument("--start_date", required=True) p.add_argument("--end_date",  
required=True) p.add_argument("--env", default="prod") return p.parse_args() def main(): args =  
parse_args() spark = SparkSession.builder.appName("my_job").getOrCreate() df =  
spark.read.parquet(args.input) .filter((F.col("date") >= args.start_date) & (F.col("date") <=  
args.end_date)) # ... transform ... df.write.mode("overwrite").delta(args.output) if __name__ ==  
"__main__": main()
```

Data Quality & Testing

Production data pipelines require automated quality checks and unit tests to catch data issues early and prevent downstream failures.

9.1 Built-in Data Quality Checks

```
# Null checks null_counts = df.select([F.count(F.when(F.col(c).isNull(),c)).alias(c) for c in
df.columns]) null_counts.show() # Duplicate check dupes = df.groupBy("id").count().filter("count
> 1") assert dupes.count() == 0, "Duplicates found" # Range / constraint checks assert
df.filter(F.col("amount") < 0).count() == 0, "Negative amounts" assert df.filter(F.col("date") >
F.current_date()).count() == 0, "Future dates" # Referential integrity orphans =
df_orders.join(df_customers, "customer_id", "left_anti") assert orphans.count() == 0, "Orders
with no customer" # Schema validation expected_cols = {"id","name","amount","date"} assert
set(df.columns) == expected_cols, f"Schema mismatch: {set(df.columns)}" # Row count threshold
cnt = df.count() assert cnt > 1000, f"Too few rows: {cnt}" assert cnt < 10_000_000,
f"Unexpectedly large: {cnt}" # Statistical checks (outlier detection) stats =
df.select(F.mean("amount"), F.stddev("amount")).collect()[0] mean_val, std_val = stats[0],
stats[1] outliers = df.filter(F.abs(F.col("amount") - mean_val) > 3 * std_val)
```

9.2 Unit Testing with pytest

```
# conftest.py import pytest from pyspark.sql import SparkSession
@pytest.fixture(scope="session") def spark(): return (SparkSession.builder .master("local[2]")
.appName("test") .config("spark.sql.shuffle.partitions","2") .getOrCreate()) #
test_transformations.py from mypackage.transforms import clean_orders def
test_clean_orders_removes_nulls(spark): data = [(1,"Alice",100.0),(2,None,50.0),(3,"Bob",None)]
df = spark.createDataFrame(data, ["id","name","amount"]) result = clean_orders(df) assert
result.count() == 1 assert result.filter(F.col("name").isNull()).count() == 0 def
test_schema(spark): df = spark.createDataFrame([(1,"A",1.0)], ["id","name","val"]) assert "id"
in df.columns assert df.schema["id"].dataType == LongType()
```

9.3 Great Expectations Integration (Databricks)

- **ge.from_pandas(pdf)** – validate a Pandas DataFrame
- **expect_column_values_to_not_be_null** – null check
- **expect_column_values_to_be_between** – range check
- **expect_column_values_to_match_regex** – pattern check
- **expect_table_row_count_to_be_between** – row count bounds
- **Checkpoint** – attach expectations to pipelines with pass/fail actions
- **Data Docs** – auto-generated HTML quality report

Security, Governance & Best Practices

10.1 Security Patterns

- **Managed Identity** – use for Databricks → ADLS; no secrets in code
- **Azure Key Vault + Databricks Secret Scope** – store all credentials in AKV
- **Unity Catalog RBAC** – fine-grained table/column/row level security
- **Column Masking** – mask PII columns per role in Unity Catalog
- **Row Filters** – restrict rows visible per user/group
- **Network** – VNet injection, private endpoints, NSG rules
- **Audit Logs** – Azure Monitor + Databricks audit logs → Log Analytics

```
\ # Row filter (Unity Catalog) spark.sql(''' CREATE ROW ACCESS POLICY region_filter AS (region
STRING) RETURN is_member('eu-team') OR region = current_user_region() ''') spark.sql("ALTER
TABLE orders ADD ROW FILTER region_filter ON (region)") # Column mask spark.sql(''' CREATE
MASKING POLICY email_mask AS (email STRING) RETURN CASE WHEN is_member('pii-team') THEN email
ELSE regexp_replace(email, '(?<=.)\.', '*') END ''') spark.sql("ALTER TABLE customers ALTER COLUMN
email SET MASKING POLICY email_mask")
```

10.2 Production Code Patterns

- **Medallion Architecture** – Bronze (raw) → Silver (cleaned) → Gold (aggregated)
- **Idempotent Writes** – always use overwrite or Delta MERGE; never append-only
- **Schema Registry** – enforce schemas at ingest; use StructType definitions
- **Config-driven pipelines** – externalise configs (JSON/YAML); avoid hardcoding
- **Logging** – use Python logging + Databricks cluster logs + Azure Monitor
- **Exception handling** – wrap job logic in try/except; emit metrics to App Insights
- **Checkpointing** – always set checkpoint location for streaming; keep in ADLS
- **CI/CD** – Azure DevOps / GitHub Actions → Databricks Asset Bundles (DAB)
- **Modular code** – split transforms, utils, readers, writers into packages
- **Documentation** – docstrings, table comments, column tags in Unity Catalog

10.3 Azure Data Engineer Exam Topics (DP-203)

Domain	Key PySpark / Databricks Topics
Design & Implement Storage	Delta Lake, ADLS partitioning, file formats
Design & Develop Processing	PySpark transforms, Structured Streaming, ADF
Secure & Monitor	Unity Catalog, AKV, RBAC, audit logs
Optimize Performance	AQE, caching, partitioning, broadcast joins
Batch & Streaming	Trigger modes, watermarks, checkpointing, sinks
Data Quality	Expectations, schema enforcement, DQ metrics

10.4 Quick-Reference Command Cheatsheet

